



TCG6223

COMPUTER GRAPHICS

LAB PREPARATION

INTRODUCTION TO OPENGL

Introduction To OpenGL

- OpenGL Architecture and Basic OpenGL Syntax
- Display Window Management Using GLUT
- Event Driven Programming Using OpenGL



OpenGL Architecture and Basic OpenGL Syntax

What is OpenGL?

- A low-level graphics rendering and imaging **library**
- A layer of **abstraction** between graphics hardware and an application program
- An API to produce high-quality, color images of **3D objects and images**.
- A **State Machine**
- Window System and Operating System independent

OpenGL State Machine

- OpenGL provides an imperative interface to a **state machine**
 - Attributes of primitives are actually states machines
 - **States are persistent**, e.g. if we set the colour, it remains the same until we change it again
 - Graphics primitives pass the state machine in a **pipeline**
 - Most functions change the state of the machine

OpenGL State Machine

- Drawing geometry and Clearing screen
- Points, Lines and Polygons
- Images and Bitmaps
- Transformations
- Colors and Shading
- Blending and Antialiasing
- Lighting and Texturing
- Hidden Surface Removal
- Display Lists
- etc

OpenGL Interface

- Components of the OpenGL Interface
 - **GL**: core OpenGL functions
 - **GLU**: graphics utility library
(helpers for creating common objects, eg. Spheres)
 - **GLUT**: GL Utility Toolkit
(interface to windowing system)
 - **GLX**: low-level interface to X Windows

Basic OpenGL Syntax

- **Function names** from GL library

- prefixed with *gl*

glBegin, glClear, glCopyPixels

- Symbolic **constants**

- Capital letters with underscore

GL_2D, GL_RGB, GL_POLYGON

- OpenGL built-in **data types**

GLbyte, GLshort, GLint, GLfloat,
GLdouble, GLboolean

Header Files

- With Microsoft Windows, the header file that accesses the WGL routines is **windows.h**.
- This header file must be listed before the OpenGL and GLU header files because it contains macros needed by the Microsoft Windows version of the OpenGL libraries.
- So the source file in this case would begin with

```
#include <windows.h>
#include <GL/gl.h>
#include <GL/glu.h>
```

Header Files

- However, if we use GLUT to handle the window-managing operations, we do not need to include **gl.h** and **glu.h** because GLUT ensures that these will be included correctly.
- Thus, we can replace the header files for OpenGL and GLU with
#include <GL/glut.h>
- In addition, we will often need to include header files that are required by the C++ code. For example,
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

Display Window Management Using GLUT

GLUT Library

- Windowing toolkit library for OpenGL
 - Hide windowing system dependencies
 - **Initialize and open** windows
 - Handle **keyboard, mouse and redraw** events
 - Enter event-driven loop, and render OpenGL

GLUT Library

- Function names from GLUT library
 - prefixed with *glut*

```
void glutInit (int argc, char ** argv)
```

- **Initializes GLUT** and should be called before any OpenGL functions.

```
void glutCreateWindow (*char title)
```

- **Creates a window** on the screen with the title given by the argument.

Event Loops & Callback Function

- **Events** – mouse, keyboard, windows events.
- **Callback function** – define how the program should react to specific events.

```
void glutDisplayFunc (void (*func) (void))
```

- The function **func()** is called each time there is a display callback

Event Loops & Callback Function

```
void glutMainLoop()
```

- Causes the program to enter an event-processing **loop**.
- Should be the **last function** in main().

Drawing A Rectangle

- The fundamental entity for specifying geometric objects is **vertex**, a location in space. Singular = Vertex (a point), Plural = Vertices
- Simple objects (eg. lines) can be specified through a collection of vertices.



```
void glVertex*(TYPE *coordinates)
```

- * allows the programmer to specify the **dimension and data type**.

Drawing A Rectangle

```
void glVertex2f(0.5, 0.5)
```

data type:

f float

d double float

s signed short integer

i signed integer

no of components / dimension (**2**, **3**, **4**)

- What is *glVertex2i* and *glVertex3f*?

Drawing A Rectangle

- Must tell OpenGL
 - What object **a list of vertices** defines
 - Where **beginning and end** of list occurs

```
void glBegin(GLenum mode)
```

- Specifies the **beginning** of an object of type mode
 - Modes: GL_POINTS, GL_LINES, etc.
-  All capital letters

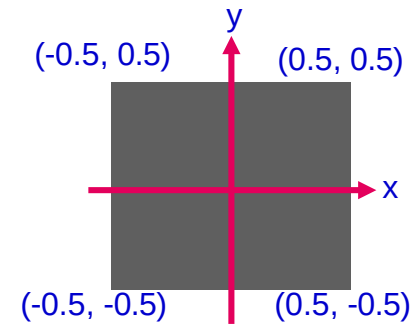
```
void glEnd()
```

- Specifies the **end** of the list of vertices

Drawing A Rectangle

- Using the three functions, we can define our rectangle as follows:

```
glBegin(GL_POLYGON);  
    glVertex2f (-0.5, -0.5);  
    glVertex2f (-0.5, 0.5);  
    glVertex2f (0.5, 0.5);  
    glVertex2f (0.5, -0.5);  
glEnd();
```



Drawing A Rectangle

- Before we draw the rectangle, we clear the color buffer.

```
void glClear (GL_COLOR_BUFFER_BIT )
```

- Clears all buffers whose bits are set in mask.
- **GL_COLOR_BUFFER_BIT** refers to the color buffer.

```
void glFlush()
```

- Forces the renderer to output results.

A Simple Program (OpenGL)

```
#include <GL\glut.h>
```

```
void display()
```

```
{
```

```
    glClear (GL_COLOR_BUFFER_BIT);
```

```
    glBegin (GL_POLYGON);
```

```
        glVertex2f (-0.5, -0.5);
```

```
        glVertex2f (-0.5, 0.5);
```

```
        glVertex2f (0.5, 0.5);
```

```
        glVertex2f (0.5, -0.5);
```

```
    glEnd ();
```

```
    glFlush ();
```

```
}
```

```
int main(int argc, char** argv)
```

```
{
```

```
    glutInit (&argc, argv);
```

```
    glutCreateWindow ("Simple");
```

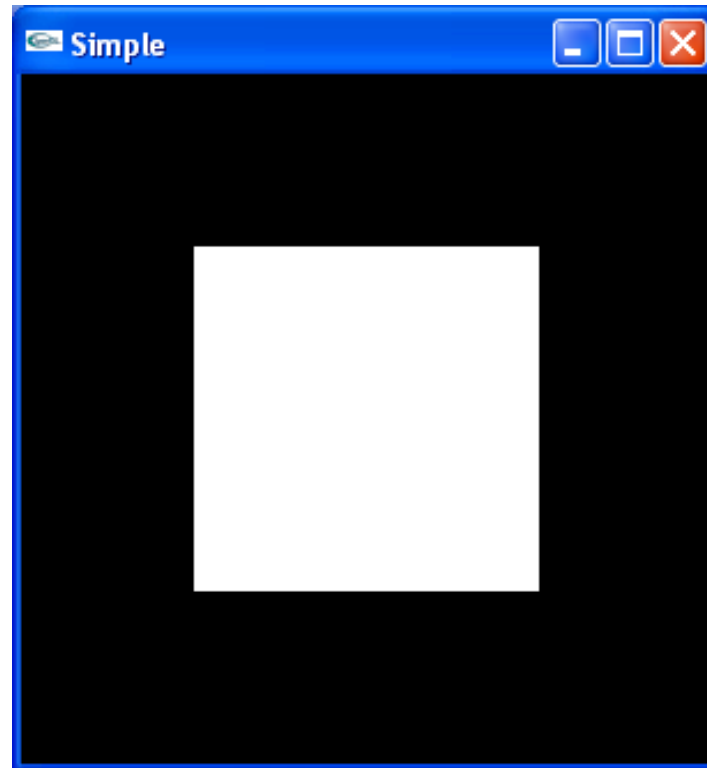
```
    glutDisplayFunc (display);
```

```
    glutMainLoop ();
```

```
    return 0;
```

```
}
```

A Simple Program



Setting Color (OpenGL)

Set a particular color:

```
glColor3f(r, g, b);
```

Set a background color:

```
glClearColor(r, g, b, 1);
```

Clear the window to background color:

```
glClear(GL_COLOR_BUFFER_BIT);
```

Using RGB Color (OpenGL)

```
#include <GL\glut.h>
```

```
void display()
```

```
{
```

```
    glClear (GL_COLOR_BUFFER_BIT);
```

```
    glColor3f(1.0, 0.0, 0.0);
```

```
    glBegin (GL_POLYGON);
```

```
        glVertex2f (-0.5, -0.5);
```

```
        glVertex2f (-0.5, 0.5);
```

```
        glVertex2f (0.5, 0.5);
```

```
        glVertex2f (0.5, -0.5);
```

```
    glEnd ();
```

```
    glFlush ();
```

```
}
```

```
int main(int argc, char** argv)
```

```
{
```

```
    glutInit (&argc, argv);
```

```
    glutCreateWindow ("Simple");
```

```
    glClearColor(1.0, 1.0, 1.0, 0.0);
```

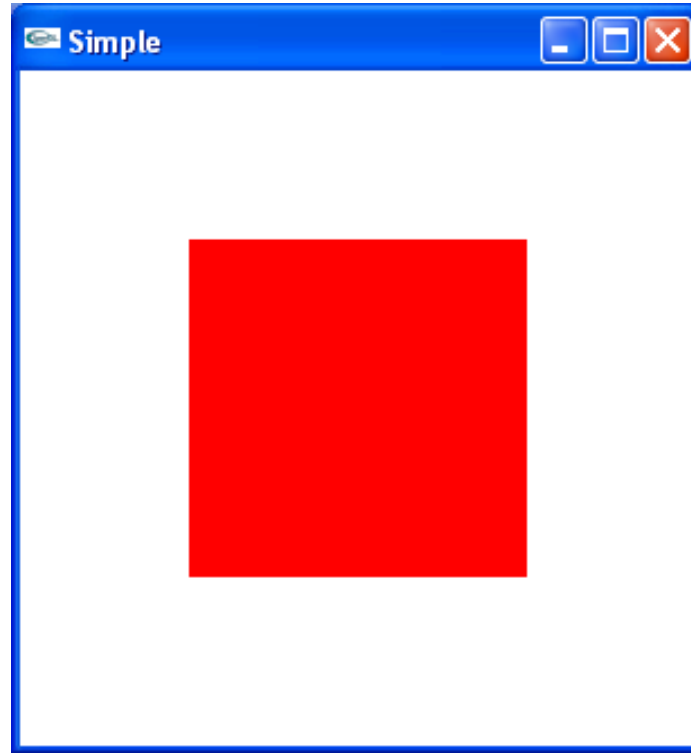
```
    glutDisplayFunc (display);
```

```
    glutMainLoop ();
```

```
    return 0;
```

```
}
```


Using RGB Color (OpenGL)



Changing GLUT Defaults

```
void glutInitDisplayMode  
                (unsigned int mode)
```

- Request a display with the properties in mode. The values of mode are combined by using the logical **OR** of options, such as **color model** (GLUT_RGB, GLUT_INDEX) and **buffering of color buffers** (GLUT_SINGLE, GLUT_DOUBLE)

Changing GLUT Defaults

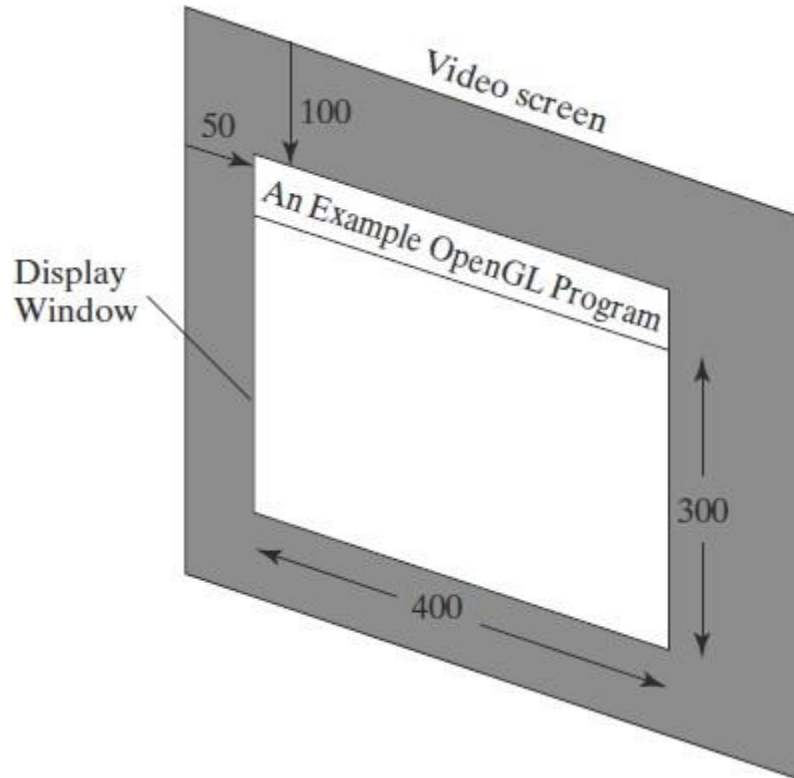
```
void glutInitWindowSize  
    (int width, int height)
```

- Specifies the **initial height and width** (in pixels) of the window on the screen.

```
void glutInitWindowPosition(int x, int y)
```

- Specifies the **top-left corner** of the window, measured in pixels, from the top-left corner of the screen.

Changing GLUT Defaults



`glutInitWindowPosition (50, 100);`

`glutInitWindowSize (400, 300);`

A 400 by 300 display window at position (50, 100) relative to the top-left corner of the video display.

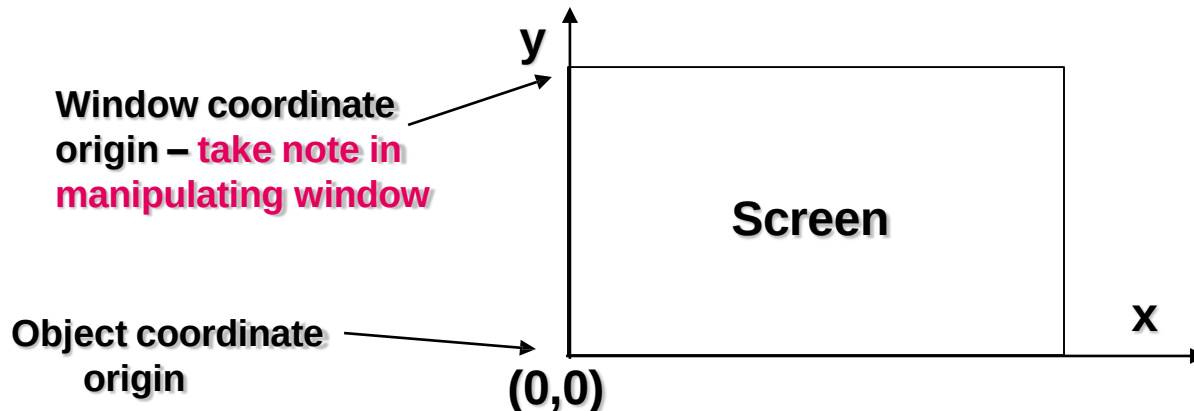
Coordinate Systems

■ Object / world coordinates

- Application **coordinates** that user use to write their program.
- Defined using **glVertex*()**.
- The origin is in the **lower left corner** of the window.

■ Window / screen coordinates

- Uses unit measured in pixels
- Allowable **range** is determined by properties of **physical display**
- The origin is in the upper left corner of the window.



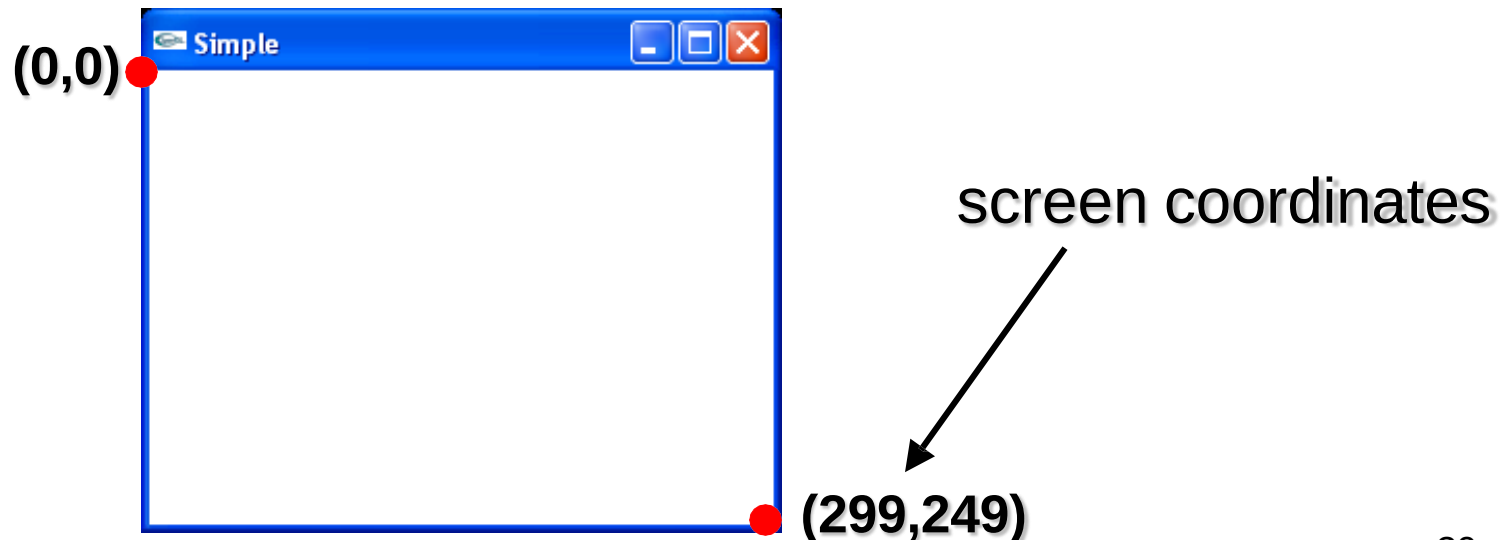
Coordinate Systems & Transformation

- Coordinate transformation from object to window is done automatically by OpenGL during rendering.

Two information are needed:

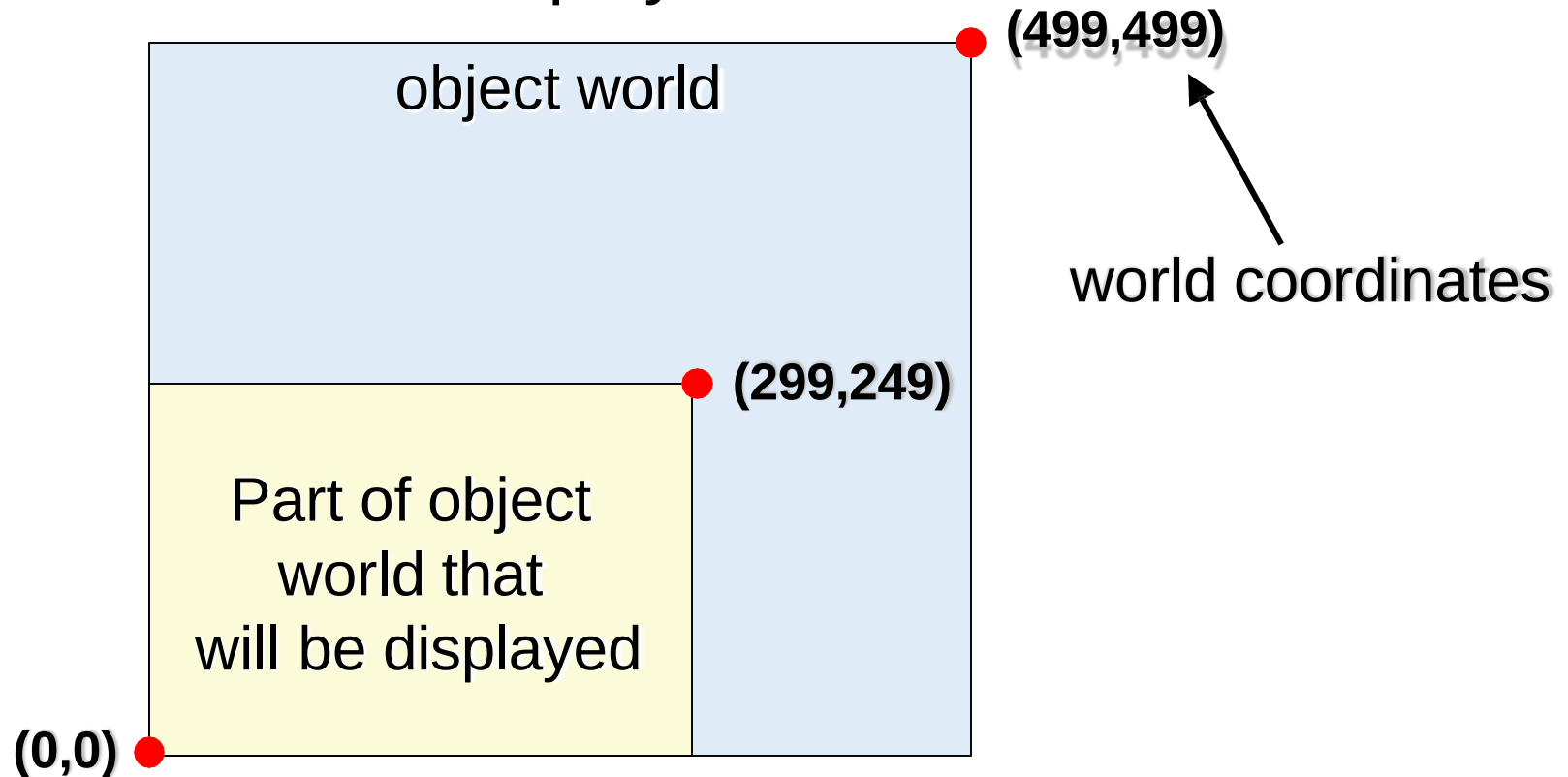
1. Size of display window

```
glutInitWindowSize(300, 250);
```



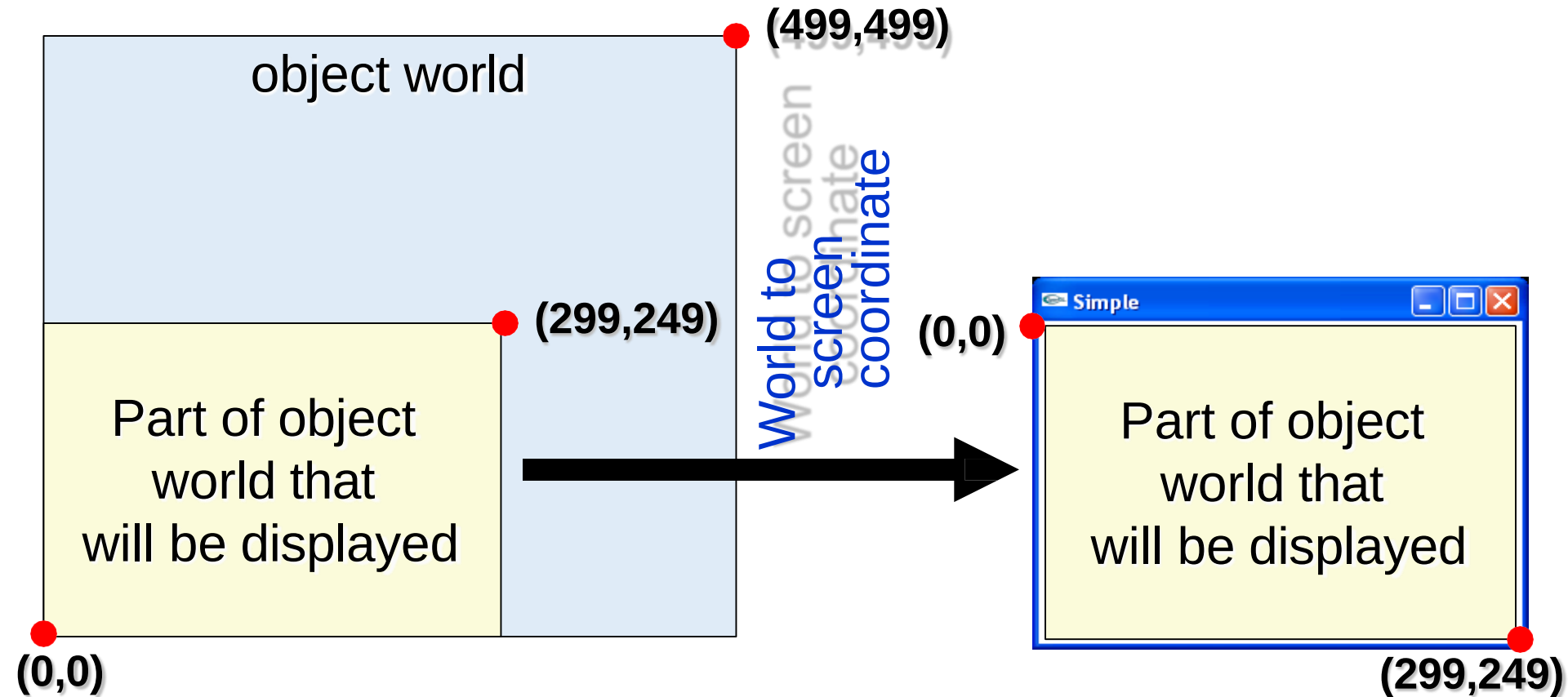
Coordinate Systems & Transformation

2. How much of the object world the user wishes to display?

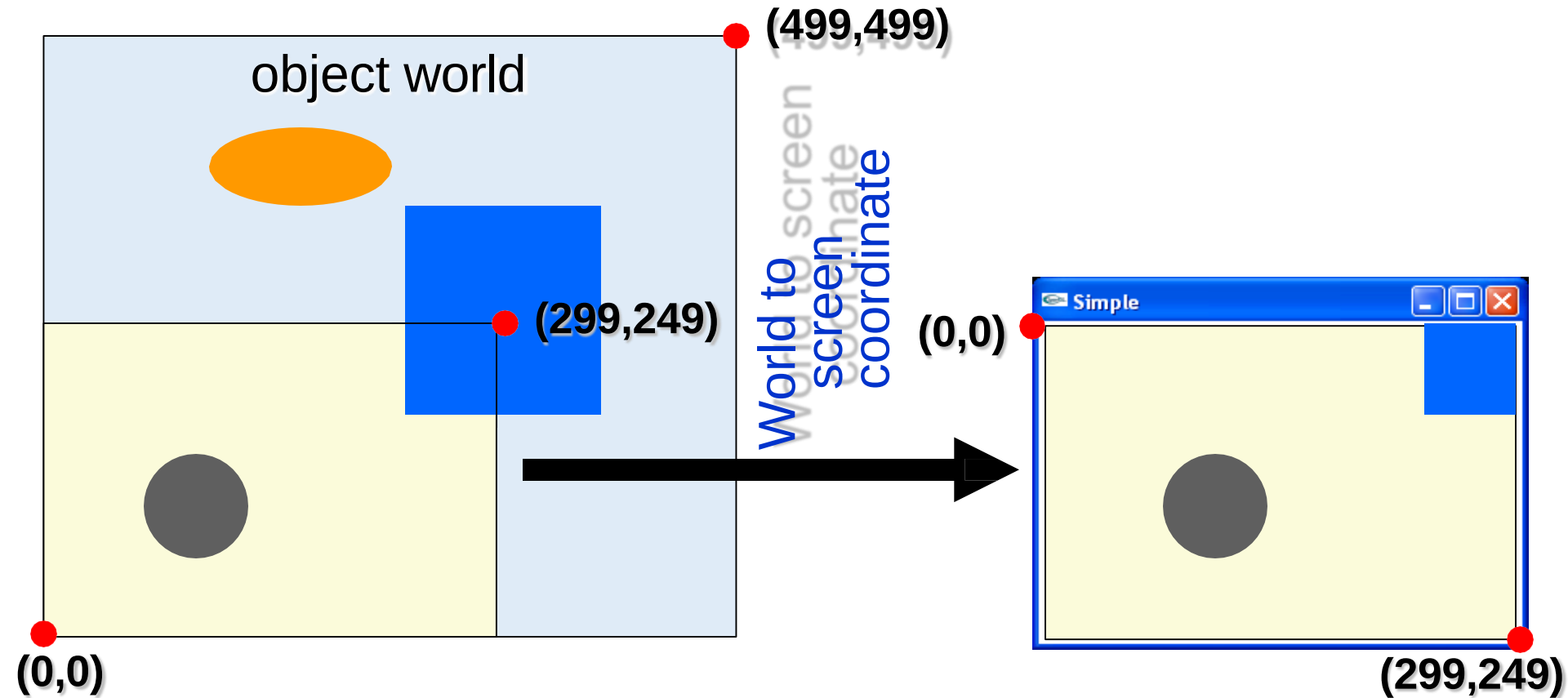


```
gluOrtho2D(0, 299, 0, 249);
```

Coordinate Systems & Transformation



Coordinate Systems & Transformation



2nd Version of a Simple Program

```
#include <GL\glut.h>
```

```
void display()
```

```
{  
    glClear (GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0, 0.0, 0.0);  
    glBegin (GL_POLYGON);  
        glVertex2f (-0.5, -0.5);  
        glVertex2f (-0.5, 0.5);  
        glVertex2f (0.5, 0.5);  
        glVertex2f (0.5, -0.5);  
    glEnd ();  
    glFlush ();  
}
```

```
void init()
```

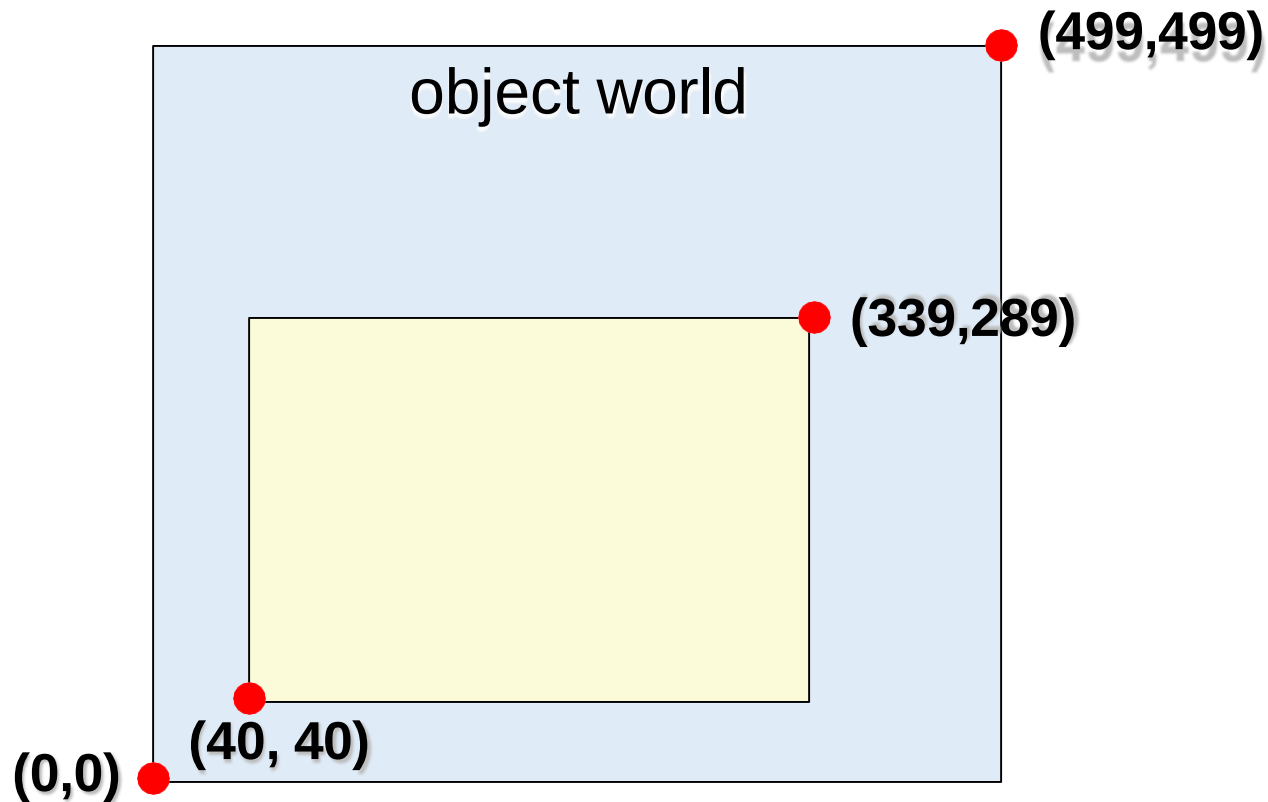
```
{  
    glClearColor(1.0, 1.0, 1.0, 0.0);  
  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);  
}
```

2nd Version of a Simple Program

```
int main(int argc, char** argv)
{
    glutInit (&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutInitWindowPosition(0, 0);
    glutCreateWindow ("Simple");
    init();
    glutDisplayFunc (display);
    glutMainLoop ();
    return 0;
}
```

Question

Can I displayed only the yellow rectangle portion on the screen? If yes, how?



```
gluOrtho2D(40.0, 339.0, 40.0, 289.0);
```

Event Driven Programming Using OpenGL

Event Driven Programming

- Mouse clicked.
- Mouse dragged.
- A key is pressed.
- The window is resized.
- The window is moved.

Event

Event Queue

- Register.
- Define.

**Call-back
Functions**

Mouse Interaction

■ Mouse Event

- Event that occurs when the mouse button is pressed or released.

■ Mouse Motion Event

- Event that occurs when the mouse is moved while one of the buttons is pressed.

Mouse Event

- Contains the following information:
 - *Button*: The mouse button that is pressed – left, middle, right.
 - *State*: The state of the button – up, down.
 - *Position*: The position of the mouse when the event occurs (x, y).

Mouse Event (OpenGL)

Registering with MouseEvent:

glutMouseFunc(myMouse);

Call-back function:

void myMouse(int button, int state, int x, int y)

Values for button:

GLUT_LEFT_BUTTON

GLUT_MIDDLE_BUTTON

GLUT_RIGHT_BUTTON

Values for state:

GLUT_UP

GLUT_DOWN

x, y:

screen coordinates of mouse position
(origin at top-left corner)

Mouse Motion Event

- Contains the following information:
 - *Position*: The current position of the mouse as the mouse is being dragged holding one of the buttons pressed. The events are continuously generated as the mouse button is pressed and dragged.

Mouse Motion Event (OpenGL)

Registering with MouseMotionEvent:

glutMotionFunc(myMovedMouse);

Call-back function:

void myMovedMouse(int x, int y)

x, y:

screen coordinates of mouse position
(origin at top-left corner)

Keyboard Event

- Contains the following information:
 - *key*: The ASCII value of the key pressed.
 - *Position*: The current position of the mouse when the key is pressed.

Keyboard Event (OpenGL)

Registering with KeyboardEvent:

glutKeyboardFunc(myKeyboard);

Call-back function:

void myKeyboard(unsigned int key, int x, int y)

Values for keys:

ASCII values for normal keys

GLUT_KEY_LEFT, GLUT_KEY_RIGHT, ... (arrow keys)

x, y:

screen coordinates of mouse position
(origin at top-left corner)

Window Events

- The **Window Redraw Event** occurs whenever the window needs to be redrawn. This happens when the window is **first opened** and when the window is exposed by **moving another window** off of it.
- The **Window Reshape Event** is generated when the window is **resized** with the mouse.

Window Events (OpenGL)

Registering with WindowRedrawEvent:

glutDisplayFunc(myDisplay);

Registering with WindowReshapeEvent:

glutReshapeFunc(Reshape);

Simple Event-Handling Program

```
#include <GL\glut.h>
#include <stdio.h>

void myDisplay(void)
{
    glClear( GL_COLOR_BUFFER_BIT );
    glFlush();
    printf("Called myDisplay\n");
}
```

```
void init()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}
```


Simple Event-Handling Program

```
void myReshape(int width, int height)
```

```
{
```

```
    printf("Called myReshape - width = %d  height = %d\n", width, height);
```

```
}
```

```
void myMouse(int button, int state, int x, int y)
```

```
{
```

```
    printf("Called myMouse - button = %c, state = %d x= %d, y = %d\n", button,  
state, x, y);
```

```
}
```

```
void myKeyboard(unsigned char key, int x, int y)
```

```
{
```

```
    printf("Called myKeyboard - key = %c, x= %d, y = %d\n", key, x, y);
```

```
}
```

Simple Event-Handling Program

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
    glutInitWindowSize(100,100);
    glutInitWindowPosition(0,0);
    glutCreateWindow("Example Event Driven");
    glutDisplayFunc(myDisplay);
    glutReshapeFunc(myReshape);
    glutMouseFunc(myMouse);
    glutKeyboardFunc(myKeyboard);
    myInit();
    glutMainLoop();
    return 0;
}
```

TCG6223

COMPUTER GRAPHICS



LAB PREPARATION
INTRODUCTION TO OPENGL